

# **LAPORAN PROJECT**

## **TRIVIA QUIZ MULTIPLAYER**

Disusun untuk memenuhi  
tugas mata kuliah Komunikasi Data A

**Disusun oleh:**

|                  |                 |
|------------------|-----------------|
| Andrea Widi      | (2408107010083) |
| Khalisha Ufairah | (2408107010084) |
| Zanna Zikraana   | (2408107010094) |



**JURUSAN INFORMATIKA**  
**FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM**  
**UNIVERSITAS SYIAH KUALA**  
**2026**

# BAB I

## PENDAHULUAN

### 1.1 Latar Belakang

Perkembangan teknologi jaringan komputer memungkinkan berbagai aplikasi dapat dijalankan secara real-time dengan melibatkan banyak pengguna secara bersamaan. Salah satu contoh penerapan komunikasi data yang banyak digunakan saat ini adalah permainan daring (online game) yang memungkinkan pemain saling berinteraksi melalui jaringan komputer. Dalam sistem seperti ini, komunikasi antara client dan server menjadi komponen utama agar data dapat dikirim dan diterima secara cepat serta teratur.

Pada mata kuliah Komunikasi Data, pemahaman mengenai konsep client-server, protokol jaringan, socket programming, serta pengiriman data secara real-time merupakan hal yang sangat penting. Oleh karena itu, diperlukan sebuah proyek yang dapat mengimplementasikan konsep-konsep tersebut dalam bentuk aplikasi yang mudah dipahami dan dapat diuji secara langsung.

Berdasarkan kebutuhan tersebut, dibuatlah aplikasi **Trivia Quiz Multiplayer**, yaitu sebuah permainan kuis berbasis jaringan yang memungkinkan beberapa pemain bergabung dan menjawab soal secara bersamaan melalui koneksi TCP/IP. Sistem ini menggunakan arsitektur client-server, di mana server bertugas mengelola permainan, mengirim soal, menghitung skor, dan menampilkan hasil akhir permainan.

Selain versi Command Line Interface (CLI), proyek ini juga menyediakan versi web yang dapat diakses melalui browser. Dengan adanya dua jenis antarmuka tersebut, pengguna dapat memahami implementasi komunikasi data baik melalui terminal maupun aplikasi berbasis web.

### 1.2 Rumusan Masalah

1. Bagaimana membangun aplikasi kuis multiplayer berbasis jaringan menggunakan konsep client-server?
2. Bagaimana menerapkan komunikasi data menggunakan protokol TCP/IP?
3. Bagaimana menghubungkan banyak client ke satu server secara bersamaan?
4. Bagaimana mengelola proses pengiriman soal, jawaban, dan skor secara real-time?
5. Bagaimana mengimplementasikan sistem permainan dalam bentuk CLI dan web-based?

### **1.3 Tujuan**

1. Membuat aplikasi Trivia Quiz Multiplayer berbasis jaringan komputer.
2. Menerapkan konsep komunikasi data menggunakan protokol TCP/IP.
3. Mengimplementasikan sistem client-server menggunakan socket programming.
4. Membuat sistem kuis yang dapat digunakan oleh beberapa pemain secara bersamaan.
5. Menampilkan hasil permainan dan skor secara real-time.
6. Memberikan contoh penerapan komunikasi data dalam aplikasi interaktif.

### **1.4 Manfaat**

1. Membantu mahasiswa memahami konsep komunikasi data secara praktis.
2. Memberikan pengalaman dalam mengembangkan aplikasi client-server.
3. Menunjukkan penerapan protokol TCP/IP pada aplikasi nyata.
4. Menjadi media pembelajaran yang interaktif mengenai jaringan komputer.
5. Menjadi contoh implementasi multi-client dalam sistem real-time.

### **1.5 Identifikasi Masalah dan Solusi**

Dalam permainan kuis tradisional, proses pengelolaan soal dan penilaian sering dilakukan secara manual sehingga kurang efisien jika jumlah peserta banyak. Selain itu, sulit untuk memastikan bahwa seluruh peserta menerima soal pada waktu yang sama.

Permasalahan lainnya adalah bagaimana menghubungkan banyak pemain ke dalam satu sesi permainan tanpa menyebabkan konflik data. Sistem juga harus mampu menghitung skor secara otomatis dan mengirimkan hasil kepada seluruh pemain.

Solusi yang diterapkan adalah membangun sistem Trivia Quiz Multiplayer berbasis TCP/IP dengan arsitektur client-server. Server bertugas mengatur jalannya permainan, mengirimkan soal kepada seluruh pemain, menerima jawaban, menghitung skor, dan menampilkan peringkat akhir. Dengan pendekatan ini, seluruh pemain dapat mengikuti permainan secara bersamaan dengan proses yang lebih terstruktur dan efisien.

## **BAB II**

### **PEMBAHASAN**

#### **2.1 Komunikasi Data**

Komunikasi data adalah proses pertukaran informasi antara dua atau lebih perangkat melalui media komunikasi tertentu. Dalam proyek Trivia Quiz Multiplayer, komunikasi data terjadi antara server dan beberapa client yang saling terhubung melalui jaringan TCP/IP.

Data yang dikirim meliputi informasi pemain, soal kuis, jawaban pemain, skor permainan, serta pesan sistem yang diperlukan selama permainan berlangsung.

#### **2.2 TCP/IP**

TCP/IP merupakan kumpulan protokol yang digunakan untuk komunikasi data dalam jaringan komputer. Pada proyek ini digunakan protokol TCP karena memiliki kemampuan menjamin data terkirim secara lengkap, berurutan, dan tanpa kehilangan paket.

Penggunaan TCP sangat sesuai untuk aplikasi kuis karena setiap soal, jawaban, dan skor harus diterima secara tepat oleh seluruh pemain agar permainan berjalan dengan adil.

#### **2.3 Client dan Server**

Server merupakan pusat pengelolaan permainan yang bertanggung jawab menerima koneksi pemain, mengatur jalannya permainan, mengirim soal, menghitung skor, dan menentukan pemenang.

Client merupakan program yang digunakan pemain untuk bergabung ke dalam permainan. Client menerima soal dari server dan mengirimkan jawaban yang dipilih oleh pemain. Hubungan client dan server pada sistem ini menggunakan model komunikasi dua arah sehingga kedua pihak dapat saling bertukar data selama permainan berlangsung.

#### **2.4 Socket Programming**

Socket merupakan antarmuka yang digunakan untuk membangun komunikasi antar perangkat dalam jaringan. Melalui socket, client dapat terhubung ke server menggunakan alamat IP dan nomor port tertentu.

Pada proyek ini digunakan socket TCP yang memungkinkan proses pertukaran data berlangsung secara stabil dan terjamin. Setiap client membuat koneksi ke server dan mengirimkan data sesuai format yang telah ditentukan.

## **2.5 Multi-Threading**

Multi-threading digunakan agar server dapat menangani banyak pemain secara bersamaan. Setiap client yang terhubung akan ditangani oleh thread terpisah sehingga aktivitas satu pemain tidak mengganggu pemain lainnya. Selain itu, server juga menggunakan thread khusus untuk mengatur jalannya permainan dan proses timer pada setiap soal.

## **2.6 Flask dan Flask-SocketIO**

Versi web pada proyek ini menggunakan framework Flask untuk membangun aplikasi berbasis browser. Flask berfungsi sebagai web server yang menyediakan antarmuka pengguna.

Flask-SocketIO digunakan untuk mendukung komunikasi real-time antara browser dan server. Dengan teknologi ini, informasi soal, skor, dan status permainan dapat diperbarui secara langsung tanpa perlu memuat ulang halaman.

## **BAB III**

### **ANALISIS DAN PERANCANGAN SISTEM**

#### **3.1 Gambaran Umum Sistem**

Trivia Quiz Multiplayer merupakan aplikasi kuis berbasis jaringan yang memungkinkan beberapa pemain mengikuti permainan secara bersamaan. Sistem terdiri atas satu server utama dan beberapa client yang terhubung melalui jaringan TCP/IP.

Server menyimpan kumpulan soal, mengelola jalannya permainan, menghitung skor, dan menampilkan hasil akhir. Client berfungsi sebagai antarmuka pemain untuk menerima soal dan mengirim jawaban.

#### **3.2 Arsitektur Sistem**

Arsitektur sistem menggunakan model client-server. Server berjalan pada port tertentu dan menunggu koneksi dari pemain. Setelah pemain berhasil bergabung, server mengirimkan informasi permainan dan memulai sesi kuis.

Komponen utama sistem meliputi:

1. Server TCP
2. Client CLI
3. Web Client
4. Flask Bridge
5. Sistem Skor
6. Timer Permainan

#### **3.3 Alur Kerja Sistem**

1. Server dijalankan dan menunggu koneksi pemain.
2. Client melakukan koneksi ke server.
3. Pemain mengirim nama untuk bergabung.
4. Server menyimpan data pemain.
5. Host memulai permainan.
6. Server mengirim soal kepada seluruh pemain.
7. Pemain mengirim jawaban.
8. Server mengevaluasi jawaban.
9. Skor diperbarui.
10. Hasil ronde dikirim ke seluruh pemain.

11. Langkah diulang hingga seluruh soal selesai.
12. Server menampilkan peringkat akhir permainan.

### **3.4 Kebutuhan Sistem**

#### **Perangkat Lunak**

1. Python
2. Visual Studio Code
3. Browser
4. Flask
5. Flask-SocketIO

#### **File Utama**

1. server.py
2. client.py
3. web\_client.py
4. requirements.txt

### **3.5 Perancangan Antarmuka**

Versi CLI menggunakan terminal sebagai media interaksi pemain. Versi web menyediakan halaman yang menampilkan daftar pemain, soal kuis, pilihan jawaban, timer, dan papan skor. Desain dibuat sederhana agar mudah digunakan oleh pengguna.

### **3.6 Cara Teknikal Pembuatan Solusi**

Sistem dibangun menggunakan bahasa pemrograman Python dengan memanfaatkan socket TCP sebagai media komunikasi utama. Server menerima koneksi dari banyak pemain dan mengelola seluruh proses permainan.

Data yang dikirim antara client dan server menggunakan format JSON sehingga lebih mudah diproses dan dikembangkan. Untuk mendukung akses melalui browser, dibuat bridge menggunakan Flask yang menghubungkan komunikasi TCP dengan antarmuka web.

Dengan kombinasi tersebut, sistem mampu menjalankan permainan kuis multiplayer secara real-time dan terstruktur.

## BAB IV

### IMPLEMENTASI SISTEM

#### 4.1 Implementasi Server

Server merupakan komponen utama dalam sistem Trivia Quiz Multiplayer. Server bertugas menerima koneksi pemain, mengelola data pemain, mengirim soal, menerima jawaban, menghitung skor, serta menentukan pemenang permainan.

Server dijalankan menggunakan perintah:

`python server.py`

Setelah dijalankan, server akan membuka port komunikasi dan menunggu client yang ingin bergabung ke dalam permainan. Setiap client yang terhubung akan disimpan dalam daftar pemain aktif.

Server juga bertanggung jawab melakukan broadcast soal kepada seluruh pemain secara bersamaan agar permainan berlangsung secara adil.





## 4.2 Implementasi Client CLI

Client CLI digunakan sebagai antarmuka pemain melalui terminal. Setelah client dijalankan, pengguna dapat memasukkan nama pemain untuk bergabung ke dalam permainan.

Client dijalankan menggunakan perintah:

`python client.py`

Setelah berhasil terhubung ke server, client akan menerima informasi pemain yang sedang aktif. Ketika permainan dimulai, soal dan pilihan jawaban akan ditampilkan pada terminal.

Pemain dapat memilih jawaban dengan mengetik huruf A, B, C, atau D sesuai pilihan yang tersedia.



### 4.3 Implementasi Web Client

Selain versi terminal, sistem juga menyediakan versi web yang dapat diakses melalui browser. Web client dibuat menggunakan Flask dan JavaScript sehingga pengguna dapat bermain dengan tampilan yang lebih interaktif.

Web client dijalankan menggunakan perintah:

```
python web_client.py
```

Setelah program berjalan, pengguna dapat membuka browser dan mengakses alamat yang telah ditentukan oleh sistem.

Melalui halaman web, pemain dapat:

1. Bergabung ke permainan.
2. Melihat daftar pemain.
3. Menerima soal secara real-time.
4. Memilih jawaban melalui tombol yang tersedia.
5. Melihat hasil ronde dan papan skor.



#### 4.4 Implementasi Protokol Komunikasi

Komunikasi antara client dan server menggunakan format data JSON. Format ini dipilih karena mudah dibaca dan diproses oleh program.

Contoh pesan yang dikirim client ke server:

```
{  
  "tipe": "GABUNG",  
  "nama": "Andrea"  
}
```

Contoh pesan jawaban:

```
{  
  "tipe": "JAWAB",  
  "jawaban": "B"  
}
```

Contoh pesan soal dari server:

```
{  
  "tipe": "SOAL",  
  "pertanyaan": "Apa fungsi DNS?",  
  "pilihan": ["A", "B", "C", "D"]  
}
```

Setiap pesan dikirim melalui koneksi TCP sehingga data dapat diterima secara lengkap dan berurutan.

#### **4.5 Implementasi Sistem Penilaian dan Timer**

Setiap soal memiliki batas waktu pengerjaan selama 15 detik. Ketika soal dikirim, server akan mengaktifkan timer yang berlaku untuk seluruh pemain.

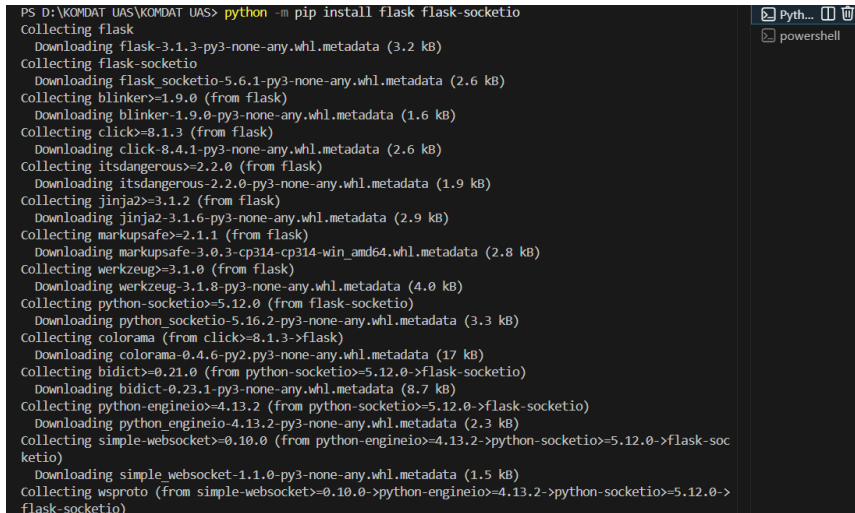
Jika pemain menjawab dengan benar, skor akan bertambah sesuai aturan permainan. Sebaliknya, jawaban yang salah tidak akan menambah skor. Setelah waktu habis, server melakukan evaluasi jawaban dan mengirimkan hasil ronde kepada seluruh pemain. Proses ini diulang hingga seluruh soal selesai.

Pada akhir permainan, server menghitung total skor seluruh pemain dan menampilkan ranking akhir berdasarkan nilai tertinggi.

## BAB V

### HASIL DAN PEMBAHASAN

#### 5.1 Hasil Pengujian Server

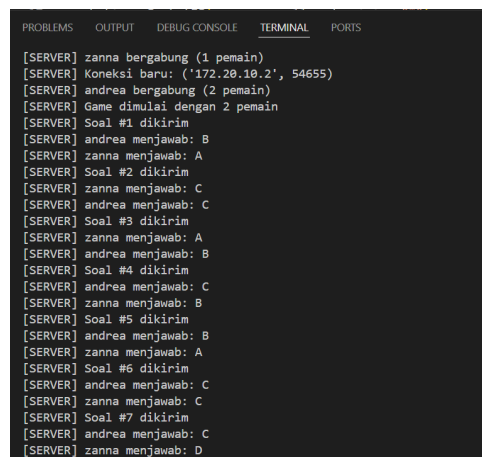


```
PS D:\KOMDAT UAS\KOMDAT UAS> python -m pip install flask flask-socketio
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting flask-socketio
  Downloading flask_socketio-5.6.1-py3-none-any.whl.metadata (2.6 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.4.1-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
  Downloading markupsafe-3.0.3-cp314-cp314-win_amd64.whl.metadata (2.8 kB)
Collecting werkzeug>=3.1.0 (from flask)
  Downloading werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Collecting python-socketio>=5.12.0 (from flask-socketio)
  Downloading python_socketio-5.16.2-py3-none-any.whl.metadata (3.3 kB)
Collecting colorama (from click>=8.1.3->flask)
  Downloading colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Collecting bidict>=0.21.0 (from python-socketio>=5.12.0->flask-socketio)
  Downloading bidict-0.23.1-py3-none-any.whl.metadata (8.7 kB)
Collecting python-engineio>=4.13.2 (from python-socketio>=5.12.0->flask-socketio)
  Downloading python_engineio-4.13.2-py3-none-any.whl.metadata (2.3 kB)
Collecting simple-websocket>=0.10.0 (from python-engineio>=4.13.2->python-socketio>=5.12.0->flask-socketio)
  Downloading simple_websocket-1.1.0-py3-none-any.whl.metadata (1.5 kB)
Collecting wsproto (from simple-websocket>=0.10.0->python-engineio>=4.13.2->python-socketio>=5.12.0->flask-socketio)
```

Gambar 5.1 Tampilan Folder Project dan Instalasi Library

Penjelasan:

Pengujian diawali dengan menyiapkan seluruh file yang dibutuhkan, seperti server.py, client.py, web\_client.py, dan README.md. Selanjutnya dilakukan instalasi library Flask dan Flask-SocketIO yang digunakan untuk mendukung komunikasi real-time pada sistem.



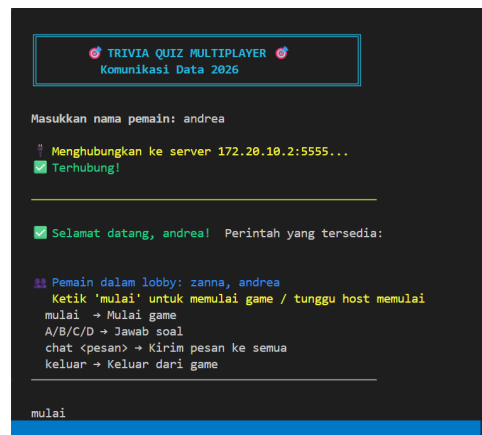
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[SERVER] zanna bergabung (1 pemain)
[SERVER] Koneksi baru: ('172.20.10.2', 54655)
[SERVER] andrea bergabung (2 pemain)
[SERVER] Game dimulai dengan 2 pemain
[SERVER] Soal #1 dikirim
[SERVER] andrea menjawab: B
[SERVER] zanna menjawab: A
[SERVER] Soal #2 dikirim
[SERVER] zanna menjawab: C
[SERVER] andrea menjawab: C
[SERVER] Soal #3 dikirim
[SERVER] zanna menjawab: A
[SERVER] andrea menjawab: B
[SERVER] Soal #4 dikirim
[SERVER] andrea menjawab: C
[SERVER] zanna menjawab: B
[SERVER] Soal #5 dikirim
[SERVER] andrea menjawab: B
[SERVER] zanna menjawab: A
[SERVER] Soal #6 dikirim
[SERVER] andrea menjawab: C
[SERVER] zanna menjawab: C
[SERVER] Soal #7 dikirim
[SERVER] andrea menjawab: C
[SERVER] zanna menjawab: D
```

Gambar 5.2 Server Berhasil Dijalankan

Penjelasan:

Server berhasil dijalankan pada localhost port 5555. Server menampilkan informasi jumlah maksimum pemain, jumlah soal, serta status menunggu pemain bergabung. Setelah pemain terhubung, server mulai mengirimkan soal dan menerima jawaban dari client hingga permainan selesai.

## 5.2 Hasil Pengujian Client CLI



```
TRIVIA QUIZ MULTIPLAYER
Komunikasi Data 2026

Masukkan nama pemain: andrea

↑ Menghubungkan ke server 172.20.10.2:5555...
✓ Terhubung!

✓ Selamat datang, andrea! Perintah yang tersedia:

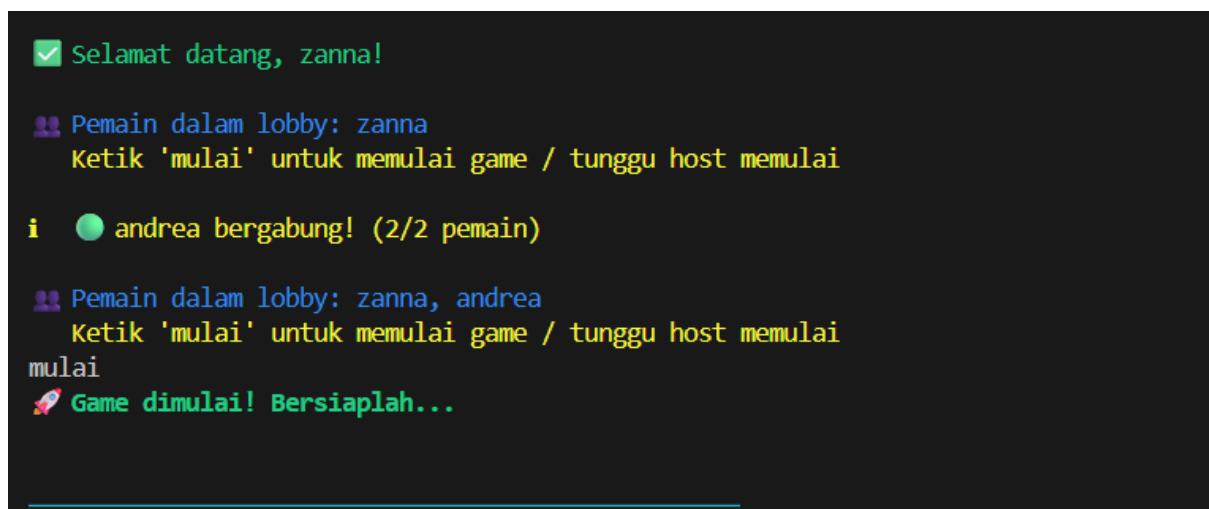
Pemain dalam lobby: zanna, andrea
Ketik 'mulai' untuk memulai game / tunggu host memulai
mulai → Mulai game
A/B/C/D → Jawab soal
chat <pesan> → Kirim pesan ke semua
keluar → Keluar dari game

mulai
```

Gambar 5.3 Client Berhasil Terhubung ke Server

Penjelasan:

Client berhasil terhubung ke server melalui alamat 127.0.0.1:5555. Setelah koneksi berhasil, pemain dapat melihat daftar perintah yang tersedia seperti memulai permainan, menjawab soal, mengirim pesan, dan keluar dari permainan.



```
✓ Selamat datang, zanna!

Pemain dalam lobby: zanna
Ketik 'mulai' untuk memulai game / tunggu host memulai

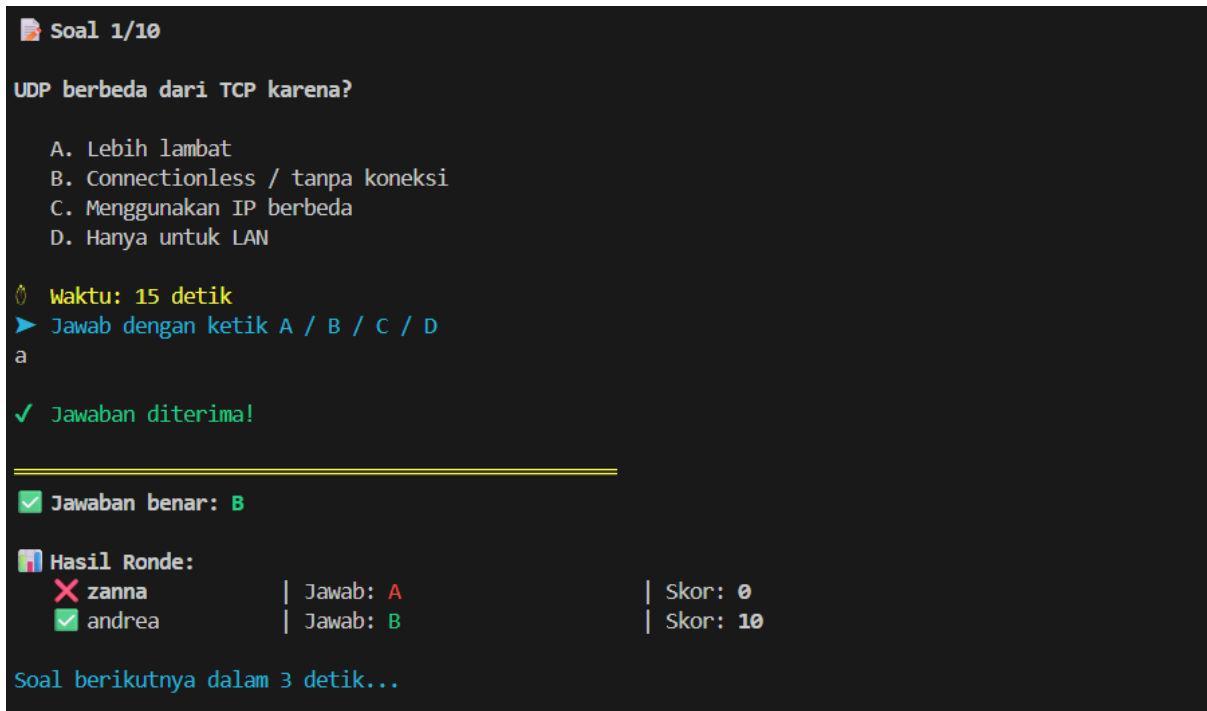
i ● andrea bergabung! (2/2 pemain)

Pemain dalam lobby: zanna, andrea
Ketik 'mulai' untuk memulai game / tunggu host memulai
mulai
🚀 Game dimulai! Bersiaplah...
```

Gambar 5.4 Pemain Bergabung ke Lobby

Penjelasan:

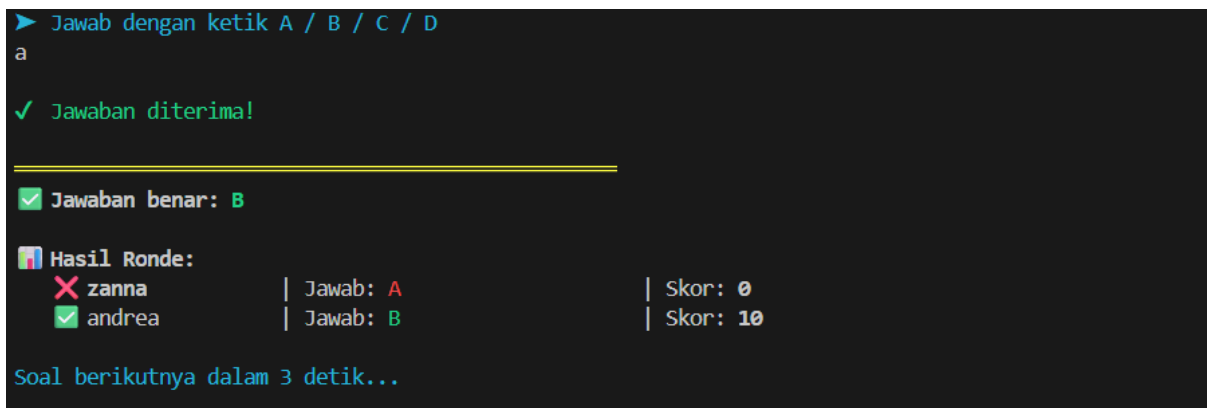
Setelah memasukkan nama pemain, sistem menampilkan daftar pemain yang berada di dalam lobby. Pemain dapat menunggu host memulai permainan atau memulai permainan secara langsung apabila bertindak sebagai host.



*Gambar 5.5 Soal Ditampilkan pada Client*

Penjelasan:

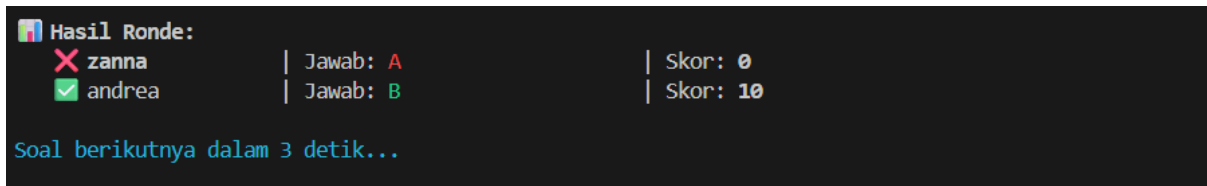
Server mengirimkan soal beserta pilihan jawaban kepada pemain. Setiap soal dilengkapi dengan batas waktu pengerjaan selama 15 detik untuk memastikan permainan berjalan secara real-time.



*Gambar 5.6 Client Mengirim Jawaban*

Penjelasan:

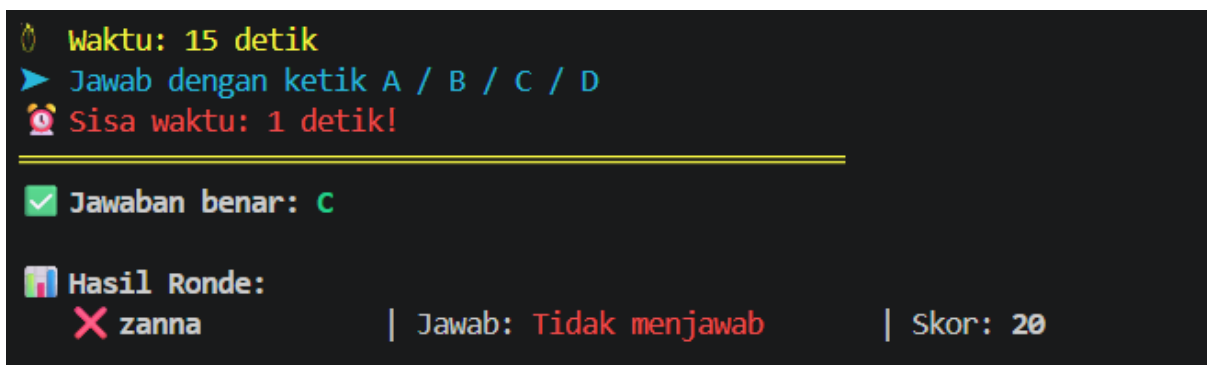
Pemain mengirimkan jawaban dengan mengetik pilihan yang tersedia. Setelah jawaban diterima server, sistem menampilkan notifikasi bahwa jawaban berhasil dikirim untuk diproses.



*Gambar 5.7 Hasil Runde Ditampilkan*

Penjelasan:

Setelah waktu habis atau seluruh pemain menjawab, server mengevaluasi jawaban dan mengirimkan hasil ronde kepada seluruh pemain. Informasi yang ditampilkan meliputi jawaban benar, jawaban pemain, dan skor sementara.



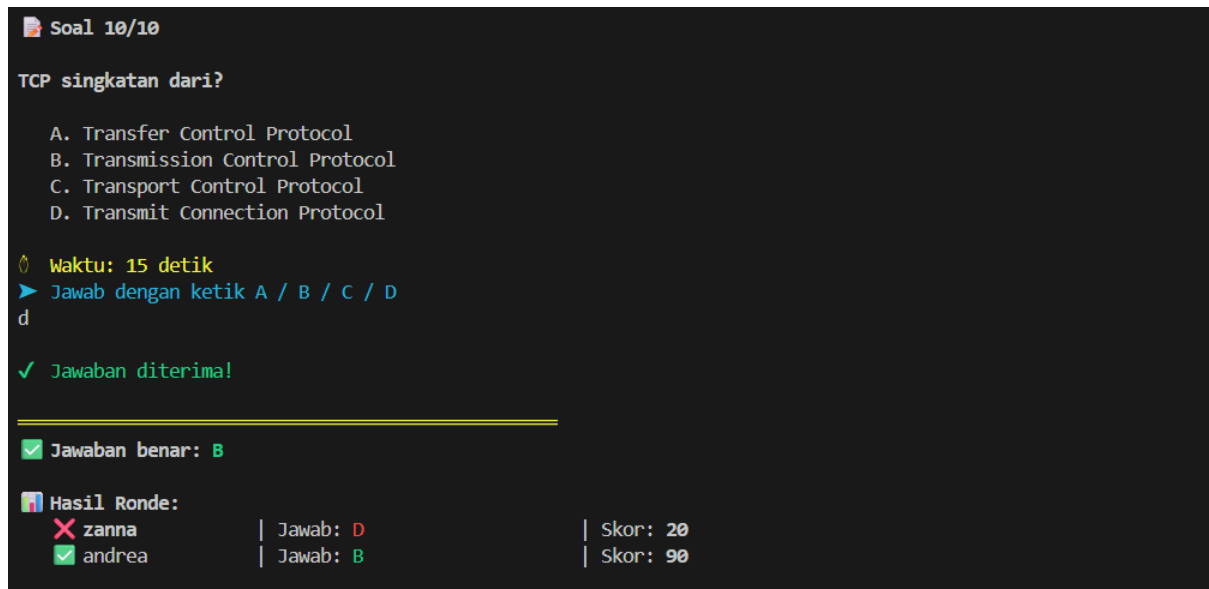
*Gambar 5.8 Contoh Jawaban Tidak Dikirim Tepat Waktu*

Penjelasan:

Apabila pemain tidak mengirim jawaban sebelum timer berakhir, sistem akan mencatat status "Tidak menjawab". Kondisi ini menunjukkan bahwa mekanisme timer berjalan dengan baik.



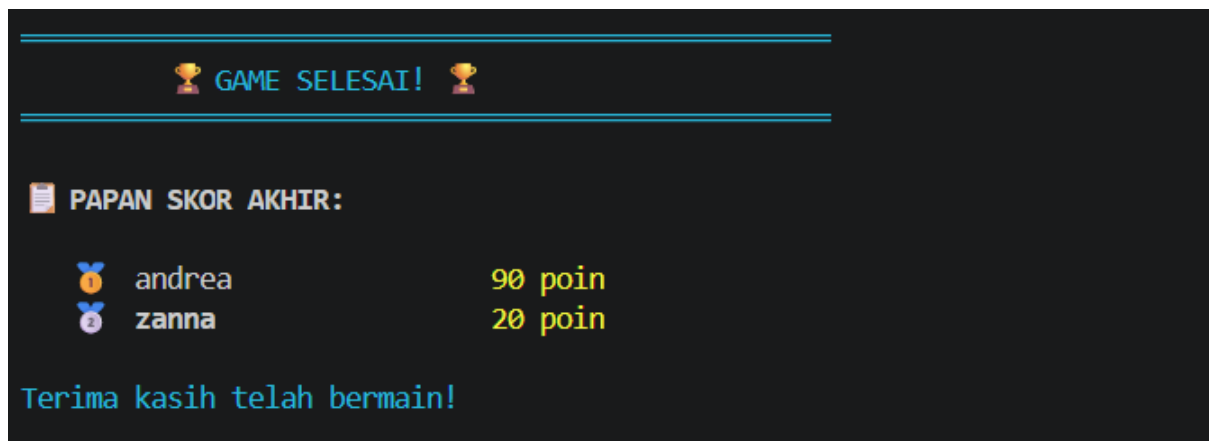
### 5.3 Hasil Pengujian Akhir Permainan



Gambar 5.9 Tampilan Soal Saat Permainan Berlangsung

Penjelasan:

Soal tetap ditampilkan selama permainan berlangsung hingga seluruh bank soal selesai diberikan kepada pemain.



Gambar 5.10 Papan Skor

Penjelasan:

Setelah seluruh soal selesai dikerjakan, server menghitung total skor pemain dan menampilkan papan skor akhir. Pada pengujian ini, pemain memperoleh skor akhir sebesar 90 poin dan 20 poin untuk masing-masing pemain dan sistem berhasil mengakhiri permainan dengan normal.

```
keluar
Sampai jumpa!

[!] Koneksi terputus: [WinError 10053] An established connection was aborted by the software in your host machineFatal
Python error: _enter_buffered_busy: could not acquire lock for <_io.BufferedWriter name='<stdout>'> at interpreter shutdown, possibly due to daemon threads
Python runtime state: finalizing (tstate=0x00007ffed172ed10)

Current thread 0x00002fb8 (most recent call first):
<no Python frame>
```

*Gambar 5.11 Koneksi Terputus Setelah Pemain Keluar*

Penjelasan:

Setelah pemain mengetik perintah “keluar”, client mengakhiri koneksi dengan server dan menampilkan pesan perpisahan. Sistem kemudian mendeteksi bahwa koneksi telah ditutup sehingga muncul notifikasi koneksi terputus. Hal ini menunjukkan bahwa mekanisme keluar dari permainan dan penghentian koneksi client telah berjalan sesuai fungsinya.

## 5.4 Pembahasan Sistem

Berdasarkan hasil pengujian yang telah dilakukan, sistem Trivia Quiz Multiplayer berhasil menerapkan konsep komunikasi data menggunakan protokol TCP/IP.

Server mampu menangani beberapa pemain secara bersamaan melalui mekanisme multi-threading. Hal ini memungkinkan permainan berlangsung secara paralel tanpa mengganggu koneksi pemain lainnya.

Penggunaan TCP memberikan keuntungan berupa pengiriman data yang lebih stabil dan terjamin. Setiap soal, jawaban, dan skor dapat diterima sesuai urutan sehingga tidak terjadi ketidaksesuaian data selama permainan.

Selain itu, implementasi Flask dan Flask-SocketIO berhasil menyediakan antarmuka web yang mampu menampilkan data permainan secara real-time.

## 5.5 Kelebihan Sistem

Kelebihan dari sistem yang dibuat adalah sebagai berikut:

1. Mendukung banyak pemain dalam satu sesi permainan.
2. Menggunakan protokol TCP yang stabil dan andal.
3. Mampu menampilkan soal secara real-time.
4. Memiliki sistem penilaian otomatis.
5. Tersedia versi CLI dan web.
6. Menerapkan konsep client-server secara lengkap.

7. Mudah digunakan untuk pembelajaran komunikasi data.
8. Mendukung pengelolaan pemain secara bersamaan menggunakan multi-threading.

## **5.6 Kekurangan Sistem**

Kekurangan dari sistem yang dibuat adalah sebagai berikut:

1. Jumlah pemain masih dibatasi.
2. Soal masih disimpan secara lokal pada server.
3. Belum menggunakan database.
4. Belum memiliki sistem login pengguna.
5. Tampilan web masih sederhana.
6. Belum tersedia fitur penyimpanan riwayat permainan.
7. Belum mendukung akses melalui internet publik.

## **5.7 Pengembangan Sistem di Masa Depan**

Beberapa pengembangan yang dapat dilakukan pada sistem ini antara lain:

1. Menambahkan database untuk penyimpanan data pemain dan skor.
2. Menambahkan fitur login dan registrasi.
3. Menambahkan kategori soal yang lebih beragam.
4. Menambahkan sistem ranking global.
5. Menambahkan fitur room atau lobby permainan.
6. Menambahkan dukungan suara dan animasi.
7. Mengembangkan aplikasi ke platform mobile.
8. Menambahkan fitur chat antar pemain.
9. Mengimplementasikan keamanan data yang lebih baik.
10. Mengembangkan sistem agar dapat diakses melalui internet.

## **BAB VI**

### **PENUTUP**

#### **6.1 Kesimpulan**

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa aplikasi Trivia Quiz Multiplayer berhasil dibangun menggunakan konsep komunikasi data berbasis client-server dengan protokol TCP/IP.

Sistem mampu menghubungkan beberapa pemain secara bersamaan, mengirimkan soal secara real-time, menerima jawaban dari pemain, menghitung skor secara otomatis, dan menampilkan hasil permainan kepada seluruh peserta.

Implementasi socket programming pada versi CLI berhasil menunjukkan proses komunikasi data secara langsung antara client dan server. Sementara itu, penggunaan Flask dan Flask-SocketIO pada versi web berhasil menyediakan antarmuka yang lebih interaktif dan mudah digunakan.

Secara keseluruhan, proyek ini telah memenuhi tujuan utama yaitu menerapkan konsep komunikasi data, TCP/IP, socket programming, multi-client, dan komunikasi real-time dalam bentuk aplikasi permainan kuis multiplayer.

#### **6.2 Saran**

Untuk pengembangan selanjutnya, sistem dapat ditingkatkan dengan menambahkan database sehingga data pemain dan hasil permainan dapat disimpan secara permanen.

Selain itu, sistem juga dapat dilengkapi dengan fitur login pengguna, manajemen room permainan, kategori soal yang lebih banyak, serta dukungan akses melalui internet agar dapat digunakan oleh lebih banyak pemain secara bersamaan.

Peningkatan pada aspek keamanan dan tampilan antarmuka juga perlu dilakukan agar sistem menjadi lebih nyaman digunakan dan mendekati standar aplikasi multiplayer yang sesungguhnya.

## DAFTAR PUSTAKA

Python Documentation. 2026. Python Socket Programming Documentation.

[Python Documentation Official Website](#)

[Flask Documentation Official Website](#)

[Flask-SocketIO Documentation Official Website](#)

Transmission Control Protocol Specification Documentation.

Internet Protocol Specification Documentation.

[Socket.IO Documentation Official Website](#)

[JSON Official Website](#)